LAPORAN TUGAS BESAR 1

Feedforward Neural Network

Disusun oleh :

Kelompok 4/K-03

Leonard Arif Sutiono	/ 18223120
Harfhan Ikhtiar Ahmad Ridzky	/ 18223123
Izhar Alif Akbar	/ 18223129

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Bagian 1. Deskripsi Persoalan....3
Bagian 2. Pembahasan....4
2.1. Penjelasan Implementasi....4
2.1.1. Deskripsi Kelas, Atribut, dan Metode....4
2.1.2. Penjelasan Forward Propagation....8
2.1.3. Penjelasan Backward Propagation dan Weight Update....9
2.2. Hasil Pengujian....10
2.2.1. Pengaruh Depth dan Width....11
2.2.2. Pengaruh Fungsi Aktivasi....12
2.2.3. Pengaruh Learning Rate....17
2.2.4. Pengaruh Regularisasi....19
2.2.5. Pengaruh Optimizer....22
2.2.6. Pengaruh Normalisasi RMSNorm....27
2.2.7. Perbandingan dengan Library Scikit-learn....29
Bagian 3. Kesimpulan dan Saran....31
3.1. Kesimpulan....31
3.2. Saran....32
Pembagian Tugas....33
Referensi....33
Lampiran Form Penggunaan AI....34

Daftar Gambar

Gambar 2.1. Pengaruh Width dengan Depth Tetap..... 13
Gambar 2.2. Pengaruh Depth dengan Width Tetap..... 14
Gambar 2.3. Grafik training loss dan validation loss untuk setiap variasi aktivasi..... 15
Gambar 2.4. Grafik distribusi parameter variasi aktivasi linear.....16
Gambar 2.5. Grafik distribusi parameter variasi aktivasi reLu..... 16
Gambar 2.6. Grafik distribusi parameter variasi aktivasi sigmoid.....17
Gambar 2.7. Grafik distribusi parameter variasi aktivasi tanh.....17
Gambar 2.8. Grafik distribusi parameter aktivasi leaky reLU..... 18
Gambar 2.9. Grafik distribusi parameter aktivasi swish..... 18
Gambar 2.10. Grafik training loss dan validation loss untuk setiap variasi learning rate..19
Gambar 2.11. Grafik distribusi bobot dan gradien variasi learning rate 0.001..... 20
Gambar 2.12. Grafik distribusi bobot dan gradien variasi learning rate 0.01..... 20
Gambar 2.13. Grafik distribusi bobot dan gradien variasi learning rate 0.05..... 21
Gambar 2.14. Grafik training loss dan validation loss untuk variasi regularisasi.....22
Gambar 2.15. Distribusi bobot dan gradien tanpa regularisasi..... 23
Gambar 2.16. Distribusi bobot dan gradien untuk regularisasi L1..... 23
Gambar 2.17. Distribusi bobot dan gradien untuk regularisasi L2..... 23
Gambar 2.18. Grafik training loss dan validation loss untuk setiap konfigurasi optimizer pada LR = 0.001..... 25
Gambar 2.19. Grafik training loss dan validation loss untuk setiap konfigurasi optimizer pada LR = 0.01..... 25
Gambar 2.20. Grafik training loss dan validation loss untuk setiap konfigurasi optimizer pada LR = 0.05..... 26
Gambar 2.21. Distribusi parameter optimizer SGD dengan LR = 0.001..... 26
Gambar 2.22. Distribusi parameter optimizer Adam dengan LR = 0.001..... 27
Gambar 2.23. Distribusi parameter optimizer SGD dengan LR = 0.01..... 27
Gambar 2.24. Distribusi parameter optimizer Adam dengan LR = 0.01..... 27
Gambar 2.25. Distribusi parameter optimizer Adam dengan LR = 0.05..... 28
Gambar 2.26. Grafik training loss dan validation loss perbandingan RMSNorm..... 29
Gambar 2.27. Distribusi parameter tanpa normalisasi..... 30
Gambar 2.28. Distribusi parameter dengan RMSNorm..... 30

Bagian 1. Deskripsi Persoalan

Feed-Forward Neural Network (FFNN) merupakan salah satu jenis dari Artificial Neural Network (ANN) yang arsitekturnya terdiri dari sebuah input layer, (beberapa) hidden layer, dan sebuah output layer. Dalam FFNN, data yang digunakan untuk pelatihan akan diolah mulai dari hidden layer pertama sampai output layer (forward propagation). Kemudian, model FFNN akan melakukan evaluasi/pembelajaran terhadap kesalahan yang dilakukan dengan iterasi terbalik dari output layer hingga hidden layer pertama (backward propagation).

Pada Tugas Besar 1 IF3270 - Pembelajaran Mesin 2025, mahasiswa diminta untuk mengimplementasikan FFNN dari awal (from scratch) menggunakan bahasa Python dengan library matematika NumPy. Dataset yang digunakan adalah Global Student Placement & Salary Dataset yang berisi data akademik mahasiswa beserta outcome penempatan kerja setelah lulus. Target yang diprediksi adalah kolom placement_status (klasifikasi biner: Placed atau Not Placed).

Pengerjaan tugas ini diharapkan dapat menambah wawasan dan pengetahuan mahasiswa mengenai arsitektur, cara kerja, serta faktor-faktor yang mempengaruhi performa FFNN.

Bagian 2. Pembahasan

2.1. Penjelasan Implementasi

2.1.1. Deskripsi Kelas, Atribut, dan Metode

Implementasi FFNN tersebar dalam beberapa file Python dengan penjelasan sebagai berikut:

1. activation.py

File ini menyimpan implementasi fungsi-fungsi aktivasi sebagai kumpulan metode statis dalam kelas Activations.

Kelas: Activations

Method	Definisi
$linear(x)$	Menghitung fungsi aktivasi linear, mengembalikan x
$linear_derivative(x)$	Turunan linear, mengembalikan array berisi 1
$reLU(x)$	Menghitung $max(0,\ x)$
$reLU_derivative(x)$	Turunan ReLU, bernilai 1 jika $x > 0$ dan 0 jika $x <= 0$
$sigmoid(x)$	Menghitung $\frac{1}{1+e^{-x}}$
$sigmoid_derivative\ (x)$	Turunan sigmoid, mengembalikan $s(1 - s)$, dengan s adalah output yang dihasilkan dari fungsi sigmoid
$tanh(x)$	Menghitung *hyperbolic tangent*, yaitu $\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
$tanh_derivative(x)$	Turunan tanh, mengembalikan $(\frac{2}{e^{x}+2^{-x}})^2$
$softmax(x)$	Untuk vektor $x = (x_1, x_2, \dots, x_n) \in \Re^n$, $softmax(x)_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$
$softmax_derivative(x)$	Turunan *softmax*, mengembalikan $s(1 - s)$, dengan s adalah output yang dihasilkan dari fungsi *softmax*.

2. loss.py

File ini mengimplementasikan fungsi-fungsi loss sebagai metode statis dalam kelas Loss.

Kelas: Loss

Method	Definisi
mse(*y_true, y_pred*)	Menghitung Mean Squared Error, $[L_{\text{MSE}} = \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2]$
mse_derivative(*y_true, y_pred*)	Turunan MSE terhadap prediksi $[\frac{\partial L_{\text{MSE}}}{\partial \hat{y}} = -\frac{2}{n}(y - \hat{y})]$
bce(*y_true, y_pred*)	Menghitung Binary Cross-Entropy dengan epsilon clipping $[L_{\text{BCE}} = -\frac{1}{n}\sum_i [y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]]$
bce_derivative(*y_true, y_pred*)	Turunan BCE terhadap prediksi $[\frac{\partial L_{\text{MSE}}}{\partial \hat{y}} = -\frac{2}{n}(y - \hat{y})]$
cce(*y_true, y_pred*)	Menghitung Categorical Cross-Entropy $[L_{\text{CCE}} = -\frac{1}{n}\sum_i \sum_j y_{ij} \log(\hat{y}_{ij})]$
cce_derivative(*y_true, y_pred*)	Turunan CCE terhadap prediksi $[\frac{\partial L_{\text{CCE}}}{\partial \hat{y}} = -\frac{1}{n} \cdot \frac{y}{\hat{y}}]$

3. initializer.py

File ini mengimplementasikan metode-metode inisialisasi bobot sebagai metode statis dalam kelas Initializer.

Kelas: Initializer

Method	Definisi

zero_initialization*(shape)*	Inisialisasi semua bobot dengan nilai 0
uniform_initialization*(shape, lower_bound, upper_bound, seed)*	Inisialisasi dengan distribusi uniform pada rentang [*lower_bound*, *upper_bound*]
normal_initialization*(shape, mean, variance, seed)*	Inisialisasi dengan distribusi normal dengan *mean* dan *variance* tertentu
xavier_initialization*(shape, n_in, n_out, seed)*	Inisialisasi Xavier (Glorot) menggunakan distribusi normal dengan mean 0 dan variansi $\sigma^2 = \frac{2}{n_{in}+n_{out}}$. Cocok untuk fungsi aktivasi Sigmoid atau Tanh.
he_initialization*(shape, n_in, n_out, seed)*	Inisialisasi He menggunakan distribusi normal dengan mean 0 dan variansi $\sigma^2 = \frac{2}{n_{in}}$. Dirancang khusus untuk fungsi aktivasi berbasis ReLU.

4. regularizer.py

File ini mengimplementasikan metode regularisasi sebagai metode statis dalam kelas Regularizer.

Kelas: Regularizer

Method	Definisi
l1(weights, lam)	Menghitung penalti L1 yaitu $\lambda sum(\lvert w \rvert)$
l1_derivative(weights, lam)	Turunan L1, mengembalikan $\lambda sign(w)$
l2(weights, lam)	Menghitung penalti L2 yaitu $\frac{\lambda}{2} sum(w^2)$
l2_derivative(weights, lam)	Turunan L2, mengembalikan λw

5. model.py

File ini mengimplementasikan dua kelas utama: Layer dan FFNN.

Kelas: Layer

Atribut:

- *n_input*: Dimensi input (integer)

- *n_output*: Dimensi output / jumlah neuron (integer)
- *activation_name*: Nama fungsi aktivasi (string)
- *W*: Matriks bobot (numpy array, shape [n_input, n_output])
- *b*: Vektor bias (numpy array, shape [1, n_output])
- *dW*: Gradien bobot (numpy array)
- *db*: Gradien bias (numpy array)
- *input_cache*: Cache input saat forward pass
- *net_cache*: Cache nilai pre-aktivasi ($net = XW + b$)
- o*utput_cache*: Cache nilai post-aktivasi

Method	Definisi
forward(x)	Menghitung *output* = $f(Wx + b)$ atau $f(RMSNorm(Wx + b))$, menyimpan *cache* untuk *backpropagation*

Kelas: FFNN

Atribut:
- layers: List dari objek Layer
- loss_name: Nama fungsi loss yang digunakan (string)
- regularization_type: Tipe regularisasi ('l1', 'l2', atau None)
- lam: Nilai lambda untuk regularisasi (float)
- history: Dictionary berisi train_loss dan val_loss per epoch

Method	Definisi
add_layer*(layer)*	• Menambahkan objek Layer ke jaringan
forward*(x)*	• Melakukan forward propagation melalui semua layer
backward*(y_true, y_pred)*	• Melakukan backward propagation menggunakan chain rule untuk menghitung dW dan db setiap layer
update_weights(learning_rate):	Memperbarui bobot menggunakan gradient descent, dengan penalti regularisasi jika diaktifkan

fit*(X_train, y_train, X_val, y_val, epochs, batch_size, learning_rate, verbose)*	Melatih model dengan mini-batch gradient descent, nilai *default* dari *batch_size* adalah 1
plot_weights_distribution*(layer_indices)*	Menampilkan histogram distribusi bobot untuk layer-layer tertentu
plot_gradients_distribution*(layer_indices)*	Menampilkan histogram distribusi gradien untuk layer-layer tertentu
save*(filename)*	Menyimpan model ke file menggunakan pickle
load*(filename)*	Memuat model dari file
initialize_weights*(method, seed, **kwargs)*	Menginisialisasi bobot semua layer dengan metode '*zero*', '*uniform*', '*normal*', '*xavier*', atau '*he*'

2.1.2. Penjelasan Forward Propagation

Forward propagation diimplementasikan dalam metode ***forward()*** pada kelas FFNN. Data mengalir secara berurutan dari *input* hingga *output* layer. Berikut adalah alur prosesnya:

1. **Inisialisasi dengan data input**

   Model menerima input ***X*** (fitur) dalam bentuk matriks batch, di mana setiap baris merupakan satu sampel data. Pada dataset yang diuji, fitur-fitur telah dilakukan ***one-hot encoding*** untuk fitur kategorikal dan ***standard scaling*** untuk fitur numerik, sehingga input memiliki dimensi **28 fitur**.

2. **Inisialisasi dengan data input**

   Untuk setiap layer, dilakukan dua operasi utama:

   - **Linear transformation**: $net = W * X + b$, di mana W adalah matriks bobot dengan *shape* [*n_input*, *n_output*] dan b adalah bias
   - **Aktivasi**: output = *f(net)*, di mana f adalah fungsi aktivasi yang telah dikonfigurasi untuk layer tersebut

   Nilai net dan output disimpan dalam cache (*net_cache* dan *output_cache*) untuk digunakan pada proses *backpropagation*.

3. **Output layer**

   Pada layer terakhir, untuk tugas klasifikasi biner, fungsi aktivasi *sigmoid* digunakan sehingga output berupa probabilitas antara 0 dan 1 untuk kelas positif (Placed).

Implementasi ***forward propagation*** pada kelas FFNN:

```
def forward(self, x):
```

```
    out = x
    for layer in self.layers:
        out = layer.forward(out)
    return out
```

Dengan ***layer.forward****(out)*:

```
def forward(self, x):
    self.input_cache = x
    self.net_cache = np.dot(x, self.W) + self.b
    activation_func = getattr(Activations, self.activation_name)
    self.output_cache = activation_func(self.net_cache)
    return self.output_cache
```

2.1.3. Penjelasan Backward Propagation dan Weight Update

Backward propagation diimplementasikan dalam metode ***backward()*** pada kelas **FFNN**. Proses ini menghitung gradien error terhadap setiap bobot menggunakan *chain rule*, dimulai dari output layer menuju input layer.

Konvensi notasi:

- W : matriks bobot
- b: bias
- net : nilai pre-aktivasi ($net = XW + b$)
- o : nilai post-aktivasi ($o = f(net)$)
- E: nilai *error*/*loss*
- δ : error term yang menyatakan $dE/dnet$ untuk sebuah layer

Langkah-langkah *Backward Propagation*:

1. **Hitung turunan loss terhadap output ($\frac{\partial E}{\partial o}$)**

   Dimulai dari output, dihitung turunan fungsi loss terhadap nilai prediksi *y_pred* menggunakan metode ***{loss_name}_derivative*** dari kelas Loss.

2. **Iterasi mundur untuk setiap layer**

   Untuk output layer: $\delta_o = \frac{\partial E}{\partial o} \cdot \frac{\partial o}{\partial net}$

   Untuk hidden layer: $\delta_h = \delta_k \cdot \Sigma W_k \cdot \frac{\partial o}{\partial net}$

   Di mana $\frac{\partial o}{\partial net}$ adalah turunan fungsi aktivasi terhadap nilai pre-aktivasi.

3. **Hitung gradien bobot dan bias**

   $\partial W = input^T \cdot \delta$ (dibagi batch size dalam *update_weights*)

∂b = *sum(delta, axis=0)*

Implementasi dalam kode:

```
def backward(self, y_true, y_pred):
    loss_derivative_func = getattr(Loss, f"{self.loss_name}_derivative")
    error_signal = loss_derivative_func(y_true, y_pred)
    for i in reversed(range(len(self.layers))):
        layer = self.layers[i]
        activation_deriv_func = getattr(Activations,
            f"{layer.activation_name}_derivative")
        da_dnet = activation_deriv_func(layer.net_cache)
        if i == len(self.layers) - 1:  # Output layer
            delta = error_signal * da_dnet
        else:  # Hidden layer
            next_layer = self.layers[i+1]
            delta = np.dot(delta_next, next_layer.W.T) * da_dnet
        layer.dW = np.dot(layer.input_cache.T, delta)
        layer.db = np.sum(delta, axis=0, keepdims=True)
        delta_next = delta
```

Selanjutnya, setelah didapatkan ΔW, dilakukan pembaruan bobot (*weight update*) menggunakan ***Gradient Descent*** standar dengan rumus:

$$W_{new} = W_{old} - \alpha(\partial W + reg_penailty)$$

$$b_{new} = b_{old} - \alpha \cdot \partial b$$

Di mana *reg_penalty* adalah penalti regularisasi ($\lambda sign(w)$ untuk L1, λw untuk L2, atau 0 jika tidak ada regularisasi). Implementasi juga menerapkan ***gradient clipping*** (*np.clip* pada *dW* dan *db* antara -5 dan 5) untuk menghindari *exploding gradient*.

Proses pelatihan (*fit*) dilakukan dengan ***mini-batch gradient descent***: (i) data *training* di-*shuffle* setiap *epoch*, kemudian dibagi menjadi *batch-batch* kecil. (ii) Setiap *batch* dilakukan ***forward pass***, ***backward pass***, dan ***weight update***. (iii) Setelah seluruh *batch* diproses, dihitung *training loss* dan *validation loss* untuk disimpan dalam histori.

2.2. Hasil Pengujian

Pada percobaan ini dilakukan pengujian menggunakan dataset ***Global Student Placement & Salary Dataset***. Konfigurasi umum yang digunakan pada semua eksperimen

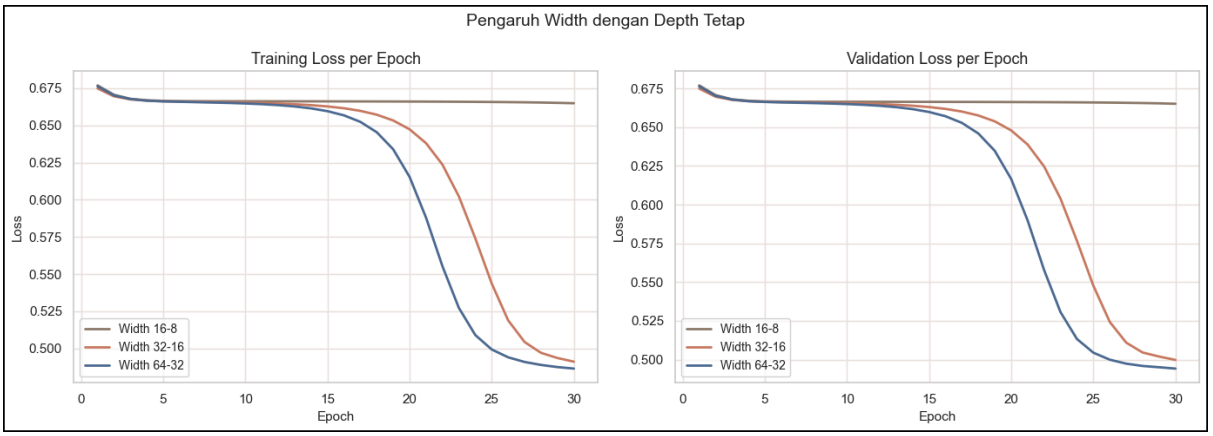
kecuali jika disebutkan lain: epochs = 30, batch_size = 32, loss = BCE, inisialisasi bobot = Uniform U(−0.1, 0.1), seed = 42.

2.2.1. Pengaruh Depth dan Width

Eksperimen dilakukan untuk menganalisis pengaruh jumlah layer (depth) dan jumlah neuron per layer (width). Aktivasi semua hidden layer menggunakan ReLU, output layer menggunakan Sigmoid, learning rate = 0,01.

Pertama, dilakukan pengujian variasi *width* (*depth* tetap = 2 *hidden layers*) terlebih dahulu:

No.	Arsitektur Hidden	Accuracy	Macro F1	Val Loss Akhir	Keterangan
1	[16, 8]	0,6155	0.761993	0.6651	Width kecil
2	[32, 16]	0,7510	0.802067	0.4997	Width sedang
3	[64, 32]	0,7495	0.801741	0.4942	Width besar



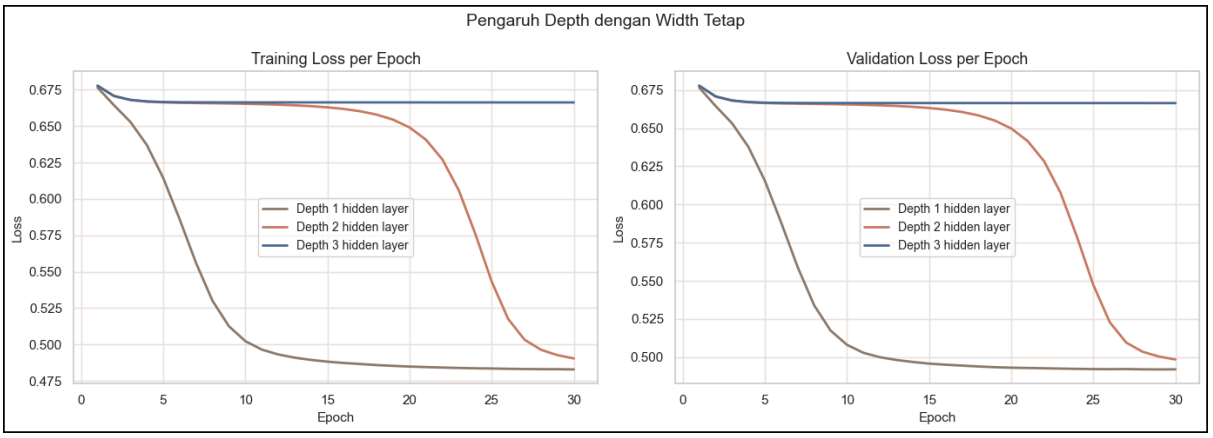
Gambar 2.1. Pengaruh Width dengan Depth Tetap

Hasil analisis variasi *width*:

- Arsitektur [16, 8] menghasilkan performa yang jauh lebih rendah (Macro F1 = 0,762) dibandingkan dua konfigurasi lainnya. Nilai val loss yang stagnan di sekitar 0,665 sepanjang 30 epoch menunjukkan bahwa model hampir tidak belajar sama sekali.
- Kegagalan arsitektur [16, 8] disebabkan oleh information bottleneck, yaitu kondisi di mana kapasitas neuron terlalu kecil untuk merepresentasikan informasi dari 28 fitur input secara memadai. Informasi yang hilang saat transformasi antar layer tidak dapat dipulihkan di layer berikutnya.
- Arsitektur [32, 16] dan [64, 32] keduanya berhasil belajar dengan baik. Loss mulai menurun tajam sekitar epoch ke-20 dan keduanya mencapai val loss di bawah 0,50 dengan Macro F1 di atas 0,80.
- Selisih performa antara [32, 16] dan [64, 32] sangat kecil (F1: 0,8020 vs 0,8017), yang mengindikasikan bahwa dataset ini tidak cukup kompleks untuk memanfaatkan kapasitas tambahan yang diberikan oleh arsitektur yang lebih lebar.

Kemudian, dilakukan pengujian *variasi depth* (width tetap = 32 neuron per layer):

No.	Arsitektur Hidden	Accuracy	Macro F1	Val Loss
1	[32]	0,7520	0,806	0,4920
2	[32, 32]	0,7535	0,806	0,4984
3	[32, 32, 32]	0,6155	0,762	0,6664



Gambar 2.2. Pengaruh *Depth* dengan *Width* Tetap

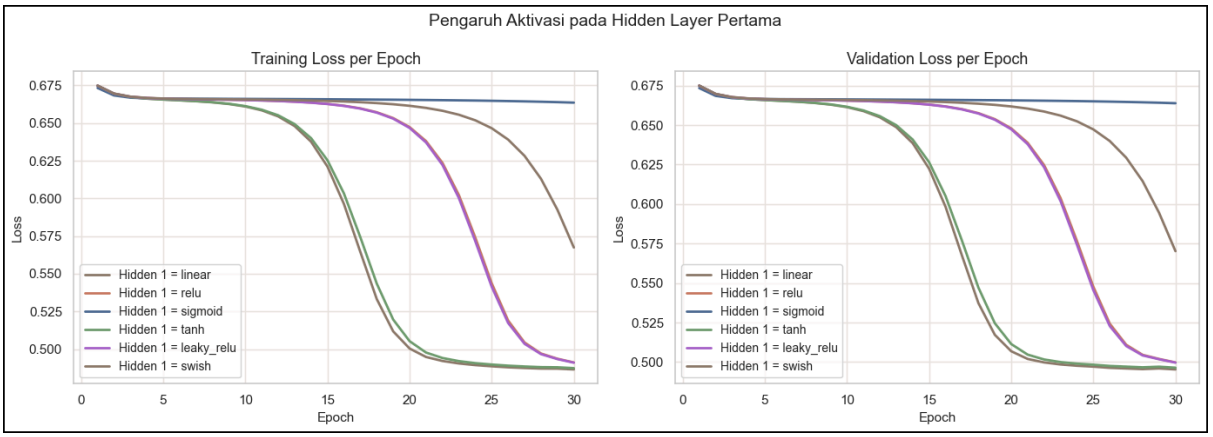
Hasil analisis:

- Arsitektur dengan 1 dan 2 hidden layer menghasilkan performa yang hampir identik (Macro F1 = 0,806), dengan kurva loss yang serupa dan konvergen secara stabil.
- Arsitektur dengan 3 hidden layer [32, 32, 32] gagal belajar, ditunjukkan oleh loss yang hampir tidak bergerak dari 0,666 selama 30 epoch (dari 0,6778 di epoch pertama hanya turun menjadi 0,6662 di epoch ke-30).
- Kegagalan arsitektur 3 hidden layer disebabkan oleh kombinasi masalah *Dying ReLU* dan *vanishing gradient*. Inisialisasi Uniform[−0,1; 0,1] akan menghasilkan bobot bernilai kecil, yang mengakibatkan banyak neuron menerima input negatif secara konsisten sehingga turunan ReLU-nya bernilai 0 secara permanen. Gradien yang bernilai 0 menyebabkan bobot tidak pernah diperbarui, dan kondisi ini semakin parah seiring bertambahnya kedalaman jaringan karena gradien harus merambat melalui lebih banyak layer.
- Hasil ini menunjukkan bahwa penambahan *depth* tanpa teknik mitigasi seperti inisialisasi **He** atau **Batch Normalization** justru berdampak negatif pada performa model untuk dataset ini.

2.2.2. Pengaruh Fungsi Aktivasi

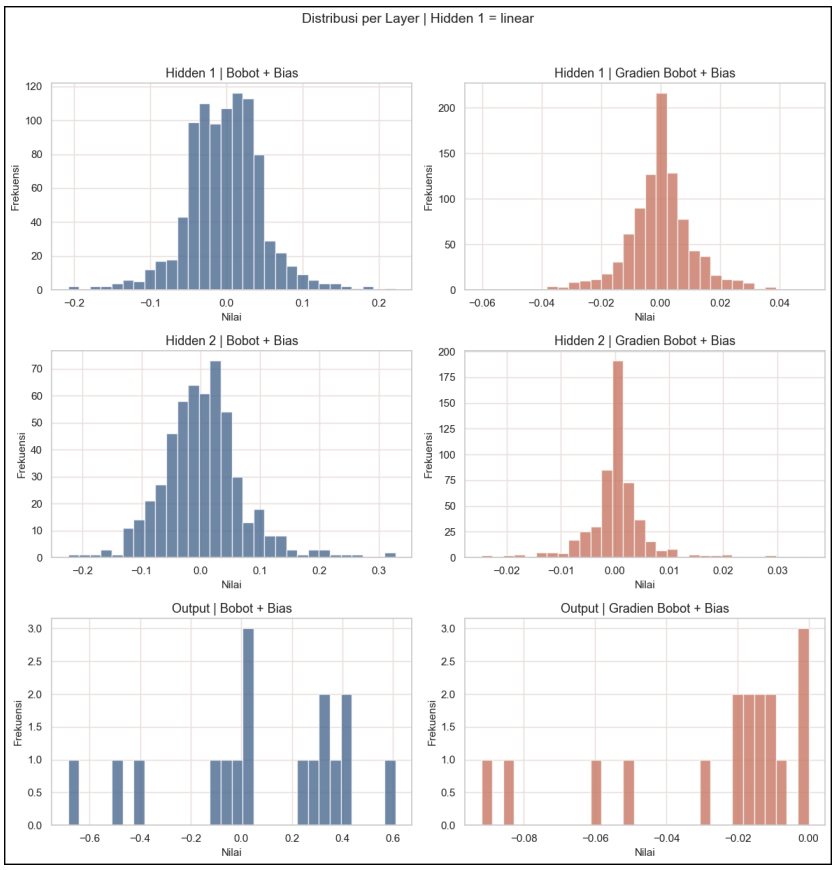
Eksperimen dilakukan dengan arsitektur [32, 16] (2 hidden layers) dan enam variasi fungsi aktivasi pada hidden layer pertama, sementara hidden layer kedua tetap menggunakan ReLU. Learning rate = 0,01, batch size = 32, 30 epoch.

No.	Aktivasi Hidden Layer 1	Accuracy	Precision	Recall	Macro F1
1	Swish	0,7325	0,710	0,955	0,815
2	Leaky ReLU	0,7515	0,785	0,820	0,803
3	ReLU	0,7510	0,785	0,820	0,802
4	Linear	0,7505	0,785	0,820	0,802
5	Tanh	0,7485	0,783	0,817	0,800
6	Sigmoid	0,6155	0,616	1,000	0,762

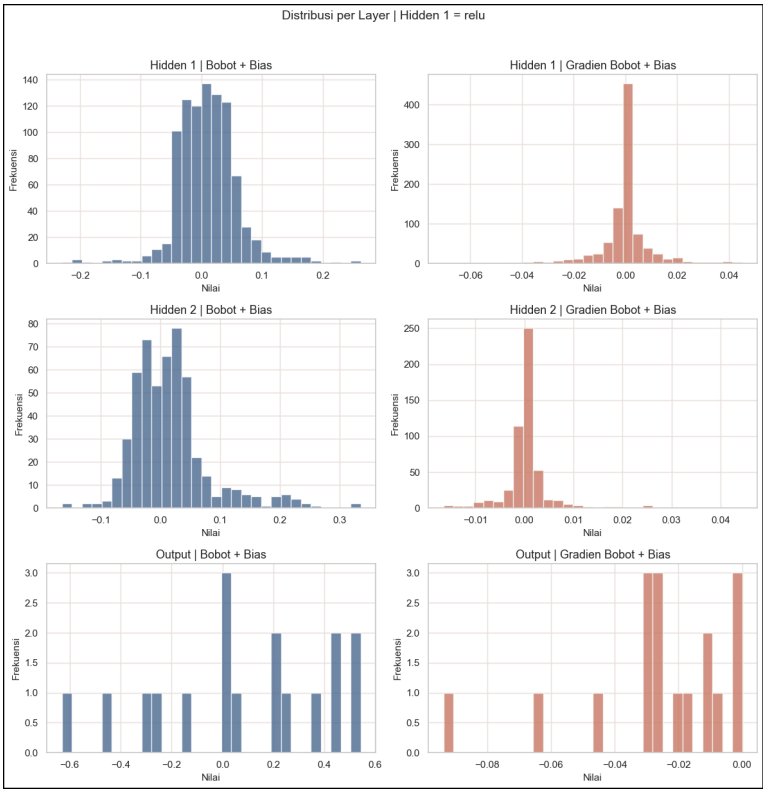


Gambar 2.3. Grafik training loss dan validation loss untuk setiap variasi aktivasi

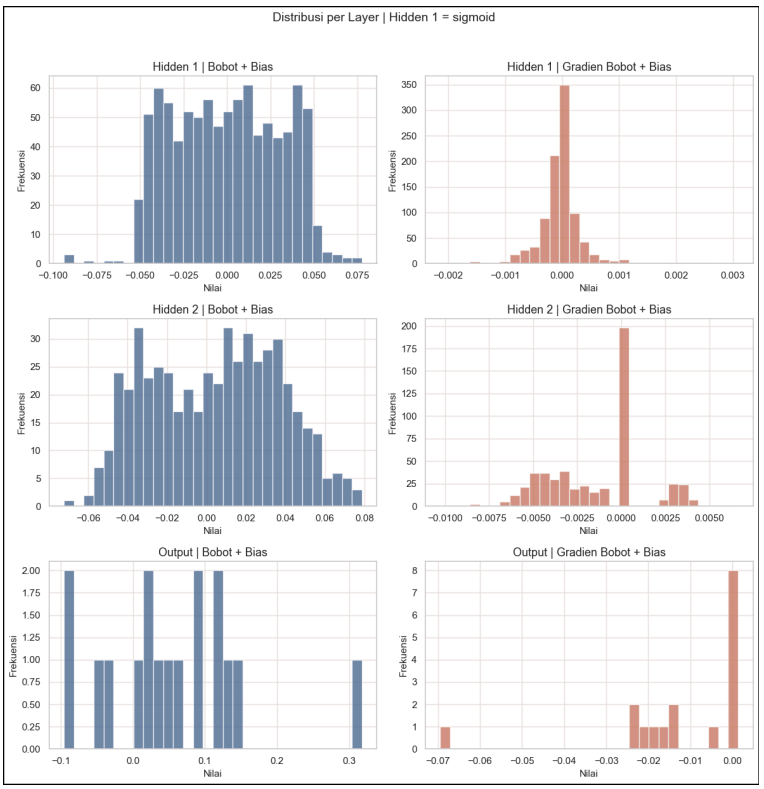
Selanjutnya, ditampilkan distribusi bobot dan gradien untuk setiap variasi aktivasi:



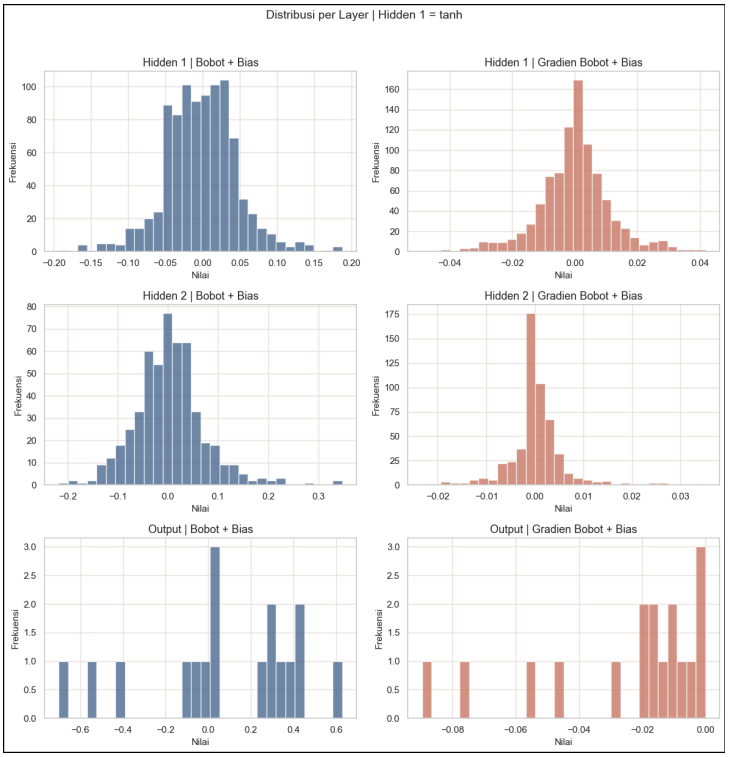
Gambar 2.4. Grafik distribusi parameter variasi aktivasi *linear*



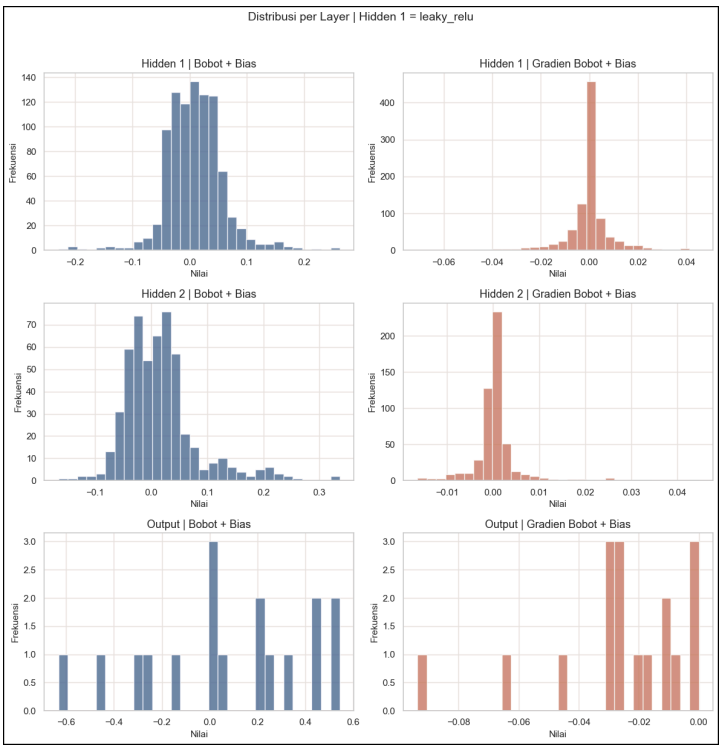
Gambar 2.5. Grafik distribusi parameter variasi aktivasi *reLu*



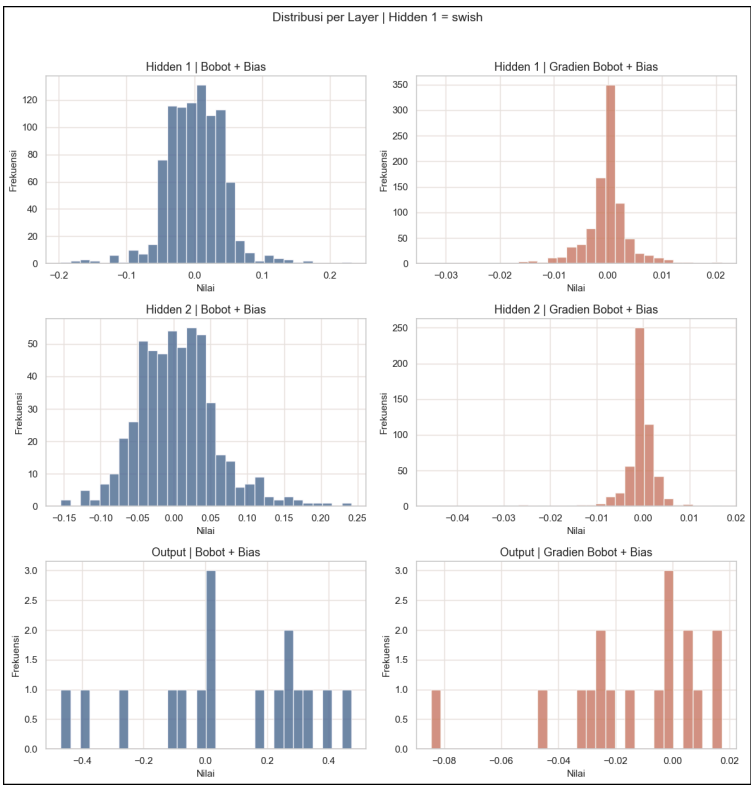
Gambar 2.6. Grafik distribusi parameter variasi aktivasi *sigmoid*



Gambar 2.7. Grafik distribusi parameter variasi aktivasi *tanh*



Gambar 2.8. Grafik distribusi parameter aktivasi *leaky reLU*



Gambar 2.9. Grafik distribusi parameter aktivasi *swish*

Hasil analisis variasi fungsi aktivasi:

- Fungsi aktivasi *Swish* menghasilkan Macro F1 tertinggi (0,815), meskipun accuracy-nya lebih rendah (0,7325) dibanding konfigurasi lain. Hal ini disebabkan

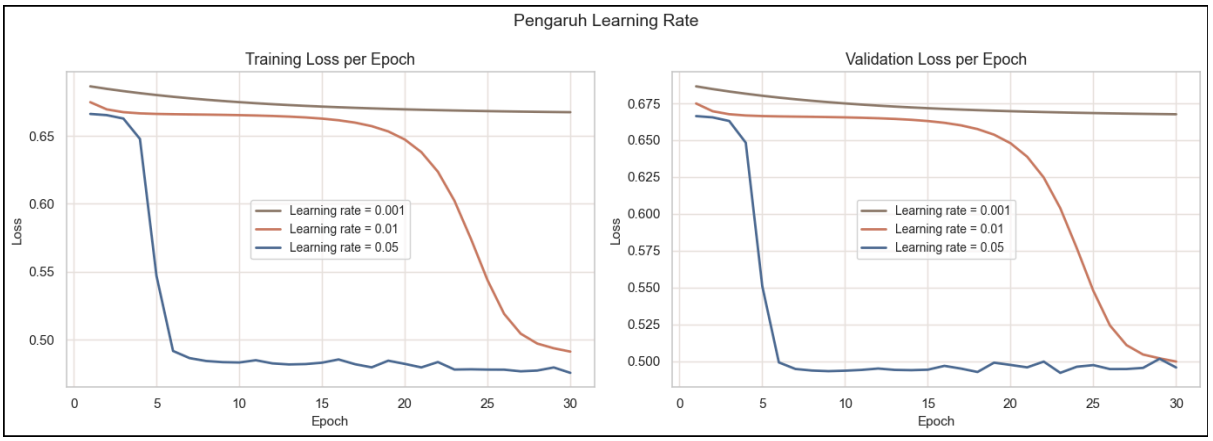
oleh nilai Recall yang sangat tinggi (0,955), artinya model berhasil mendeteksi hampir semua sampel kelas positif (Placed). Swish memiliki sifat *non-monotonic* dan *smooth* (tidak memiliki titik yang tidak dapat diturunkan), sehingga gradiennya mengalir lebih lancar selama pelatihan.

- Fungsi aktivasi *Sigmoid* pada hidden layer gagal untuk belajar (mengurangi *loss*), ditunjukkan oleh **Recall = 1,000** yang mengindikasikan model hanya memprediksi satu kelas saja. Hal ini disebabkan oleh *vanishing gradient* yang berlebih, di mana turunan sigmoid bernilai sangat kecil (mendekati 0) untuk sebagian besar rentang input, sehingga gradien yang merambat balik melalui *hidden layer beractivation sigmoid* menjadi terlalu kecil untuk memperbarui bobot secara efektif.
- Leaky ReLU, ReLU, Linear, dan Tanh semuanya berhasil belajar dengan baik dan menghasilkan performa yang serupa di rentang Macro F1 0,800 hingga 0,803. Perbedaan yang sangat kecil ini menunjukkan bahwa keempat fungsi aktivasi tersebut memiliki kemampuan representasi yang setara untuk dataset ini.
- Pilihan fungsi aktivasi sebaiknya disesuaikan dengan kebutuhan aplikasi. Apabila tujuan adalah memaksimalkan deteksi kelas positif (misalnya meminimalkan kandidat yang terlewat), *Swish* menjadi pilihan yang lebih unggul. Apabila tujuan adalah keseimbangan antara precision dan recall, *Leaky ReLU* atau *ReLU* lebih disarankan.

2.2.3. Pengaruh Learning Rate

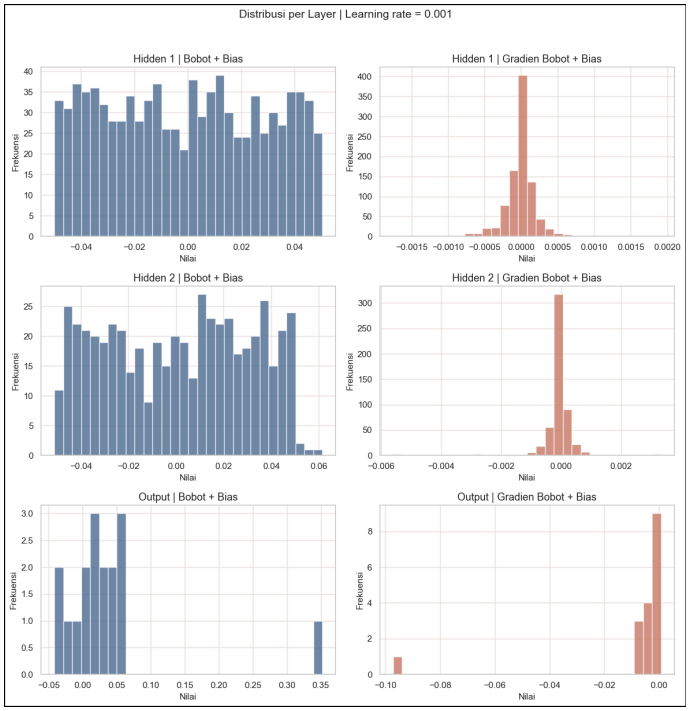
Eksperimen menggunakan arsitektur [32, 16] dengan aktivasi ReLU. Tiga nilai learning rate diuji. Hyperparameter lain tetap: batch size = 32, 30 epoch.

No.	Learning Rate (α)	Accuracy	Macro F1	Keterangan
1	0,1	0,7510	0,802	Terbaik
2	0,05	0,7495	0,800	Mendekati terbaik
3	0,001	0,6155	0,762	Tidak belajar

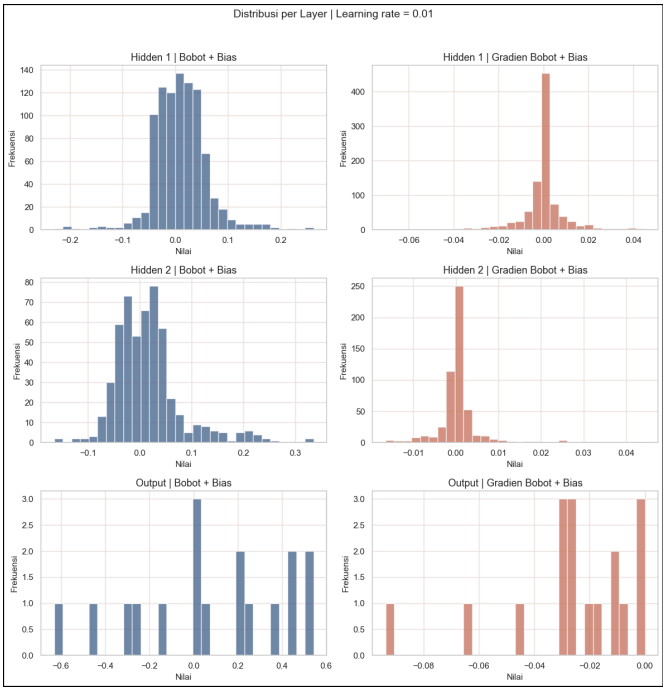


Gambar 2.10. Grafik *training loss* dan *validation loss* untuk setiap variasi *learning rate*

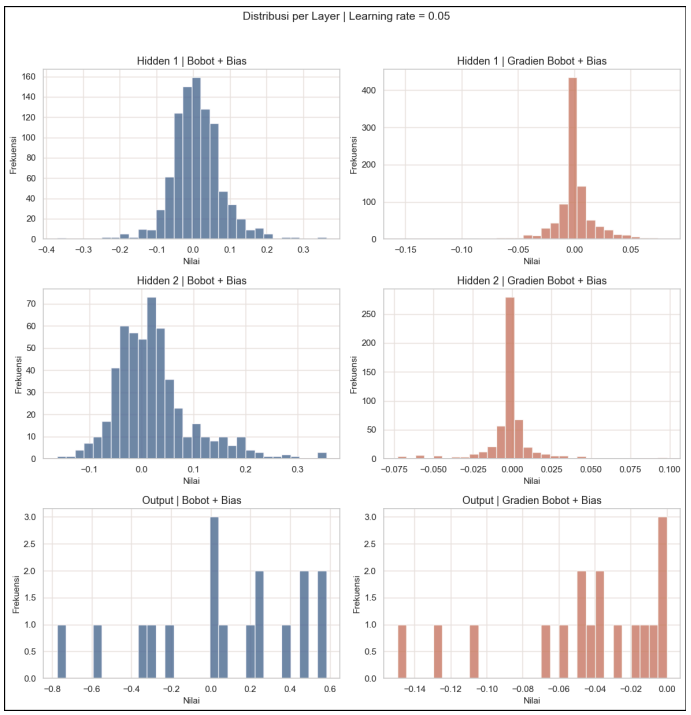
Selanjutnya, ditampilkan distribusi bobot dan gradien untuk setiap variasi *learning rate*:



Gambar 2.11. Grafik distribusi bobot dan gradien variasi *learning rate* 0.001



Gambar 2.12. Grafik distribusi bobot dan gradien variasi *learning rate* 0.01



Gambar 2.13. Grafik distribusi bobot dan gradien variasi *learning rate* 0.05

Hasil analisis variasi *learning rate*:

Pembaruan bobot pada gradient descent mengikuti rumus berikut.

$$W_{new} = W_{old} - \alpha(\partial W + reg_penailty)$$

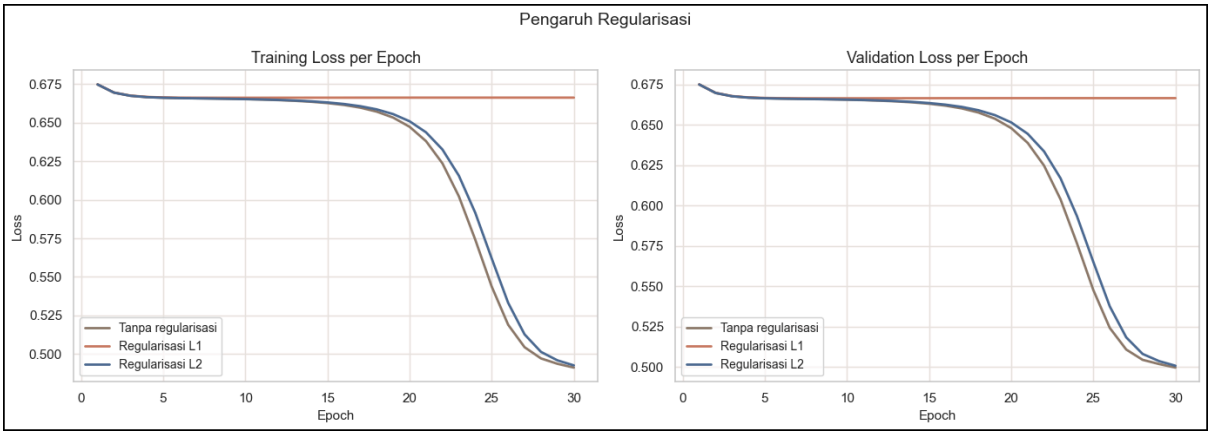
Parameter learning rate α menentukan seberapa besar langkah pembaruan pada setiap iterasi. Hasil analisis variasi learning rate adalah sebagai berikut.

- **α = 0,001** menyebabkan model gagal belajar dalam 30 epoch, ditandai oleh *val loss* yang tidak turun secara berarti dan **Recall = 1,000**. Pembaruan bobot yang terlalu kecil di setiap iterasi menyebabkan model tidak mampu bergerak secara signifikan dari posisi inisialisasi awalnya dalam jumlah epoch yang terbatas.
- **α = 0,01** memberikan hasil terbaik (**Macro F1 = 0,802**). Nilai ini merupakan titik keseimbangan antara kecepatan konvergensi yang memadai dan stabilitas pembaruan bobot.
- **α = 0,05** menghasilkan performa yang sedikit lebih rendah (Macro F1 = 0,800) dibandingkan α = 0,01. Meskipun konvergensinya lebih cepat di awal pelatihan, langkah pembaruan yang lebih besar berpotensi mengakibatkan osilasi di sekitar nilai *minimum loss*, sehingga model tidak dapat mencapai titik optimal yang sama.
- Hasil ini menegaskan bahwa pemilihan *learning rate* perlu mempertimbangkan kapasitas model dan jumlah *epoch* yang tersedia. *Learning rate* yang terlalu kecil membutuhkan lebih banyak epoch untuk konvergen, sedangkan learning rate yang terlalu besar berisiko melewati titik minimum (*overshooting*).

2.2.4. Pengaruh Regularisasi

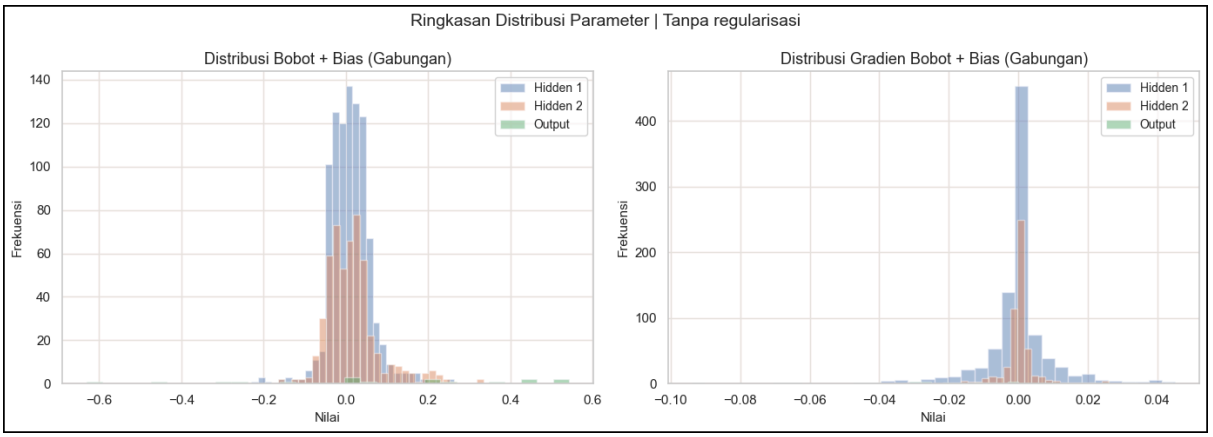
Eksperimen menggunakan arsitektur [32, 16] dengan aktivasi ReLU, learning rate = 0,01, batch size = 32, 30 epoch, dan nilai lambda (λ) = 0,001.

No.	Regularisasi	Accuracy	Precision	Recall	Macro F1
1	L2 (λ = 0,001)	0,7520	0,784	0,824	0,803
2	Tanpa Regularisasi	0,7510	0,785	0,820	0,802
3	L1 (λ = 0,001)	0,6155	0,616	1,000	0,762

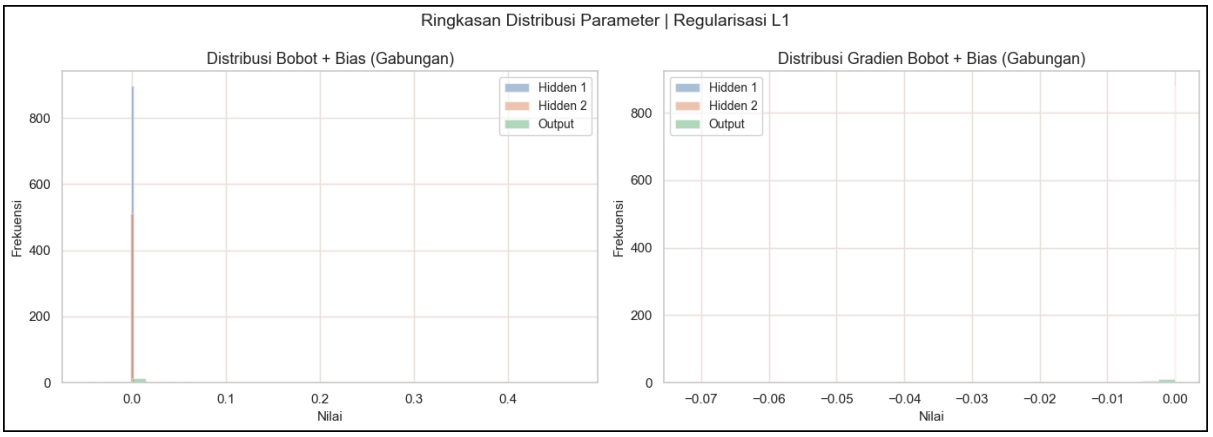


Gambar 2.14. Grafik *training loss* dan *validation loss* untuk variasi regularisasi

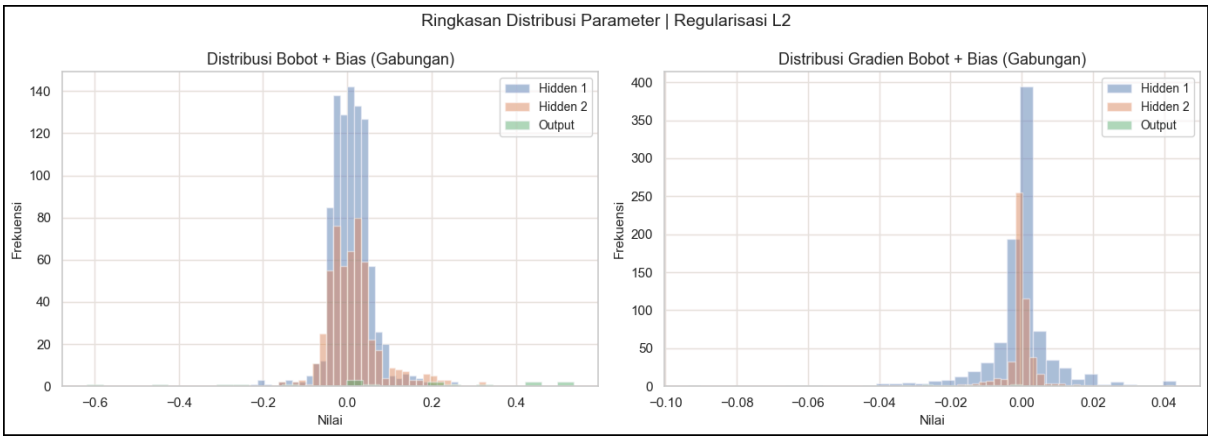
Berdasarkan kurva loss pada Gambar 2.1.4., L2 dan konfigurasi tanpa regularisasi menunjukkan pola konvergensi yang sangat serupa, dengan *training loss* dan *validation loss* yang berjalan berdekatan dan konvergen secara stabil. Hal ini mengindikasikan bahwa pada kasus ini tidak terdapat kecenderungan *overfitting* yang signifikan pada arsitektur yang digunakan. L1 menunjukkan kurva yang sepenuhnya datar, mengonfirmasi kegagalan model dalam belajar.



Gambar 2.15. Distribusi bobot dan gradien tanpa regularisasi



Gambar 2.16. Distribusi bobot dan gradien untuk regularisasi L1



Gambar 2.17. Distribusi bobot dan gradien untuk regularisasi L2

Berdasarkan distribusi bobot pada Gambar 2.15., L1 menghasilkan distribusi bobot yang sangat terpusat di sekitar nol (sparse weights), mencerminkan efek *sparsity* yang memang menjadi karakteristik utama regularisasi L1. L2 menghasilkan distribusi yang lebih menyebar dibanding L1, namun dengan nilai absolut bobot yang sedikit lebih kecil dibandingkan konfigurasi tanpa regularisasi, yang mencerminkan efek *weight decay* dari L2. Konfigurasi tanpa regularisasi menghasilkan distribusi bobot yang paling tersebar di antara ketiga konfigurasi.

Berdasarkan distribusi gradien pada Gambar 2.16., L1 menunjukkan distribusi gradien yang sangat terpusat di nol, mengonfirmasi bahwa proses pembaruan bobot terhenti akibat weight collapse. L2 dan konfigurasi tanpa regularisasi menunjukkan distribusi gradien yang serupa, namun L2 memiliki variansi gradien yang sedikit lebih kecil, yang konsisten dengan efek smoothing dari penalti kuadratik yang mendorong bobot menuju distribusi yang lebih teratur.

Hasil analisis:

- L2 menghasilkan Macro F1 terbaik (0,803) dengan peningkatan tipis atas konfigurasi tanpa regularisasi (0,802). Efek weight decay dari L2 sedikit meningkatkan generalisasi tanpa mengorbankan kapasitas model, terkonfirmasi dari distribusi bobot yang lebih teratur dan variansi gradien yang lebih kecil.

- Tidak ditemukannya gap signifikan antara *training loss* dan *validation loss* pada konfigurasi tanpa regularisasi menunjukkan bahwa dataset ini tidak rentan terhadap overfitting pada arsitektur yang digunakan, sehingga manfaat regularisasi pada konteks ini memang terbatas.
- L1 menyebabkan kegagalan belajar total (**Recall = 1,000**), temuan ini terkonfirmasi dari distribusi bobot yang kolaps ke nol dan distribusi gradien yang sepenuhnya terpusat di nol. Tekanan konstan dari turunan L1 yang bernilai $\lambda \cdot sign(w)$ mempercepat *Dying ReLU* hingga model kehilangan seluruh kapasitas belajarnya. L1 memerlukan nilai λ yang jauh lebih kecil atau teknik mitigasi tambahan apabila digunakan bersama *ReLU* dan inisialisasi *Uniform*.

2.2.5. Pengaruh Optimizer

Eksperimen membandingkan dua algoritma optimasi, yaitu Stochastic Gradient Descent (SGD) standar dan Adam, pada tiga nilai learning rate yang berbeda. Arsitektur yang digunakan adalah [32, 16] dengan aktivasi ReLU, batch size = 32, 30 epoch.

Perbedaan mendasar antara SGD dan Adam terletak pada cara pembaruan bobot dilakukan. SGD memperbarui semua bobot dengan learning rate yang sama dan tetap sepanjang pelatihan sesuai rumus berikut.

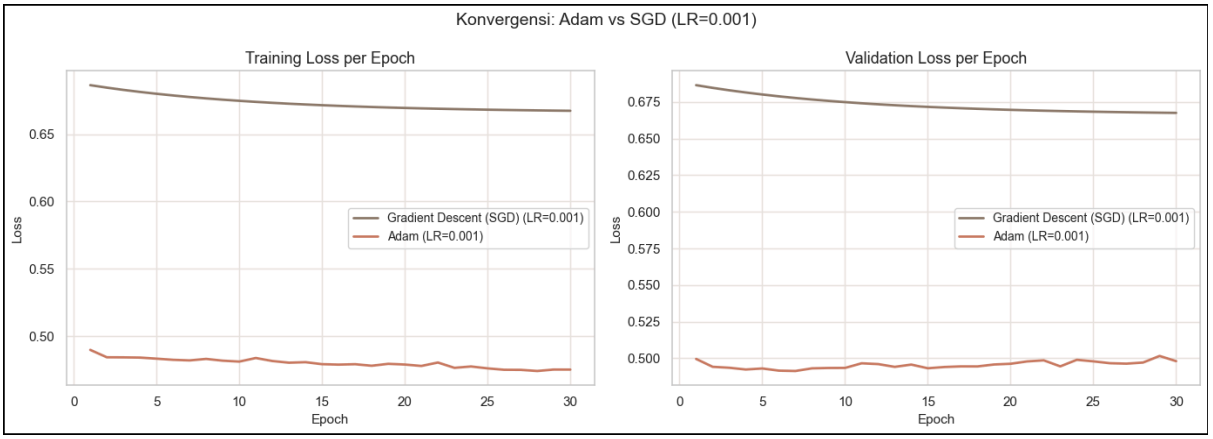
$$W_{new} = W_{old} - \alpha \cdot \frac{\partial E}{\partial W}$$

Adam, di sisi lain, mengadaptasi learning rate secara individual untuk setiap parameter berdasarkan estimasi momen pertama (rata-rata gradien, m_t) dan momen kedua (rata-rata kuadrat gradien, v_t) sebagai berikut.

$$W_{new} = W_{old} - \alpha \cdot \frac{\widehat{m_t}}{\sqrt{\widehat{v_t} + \epsilon}}$$

Hasil pada LR = 0,001

Optimizer	Accuracy	Precision	Recall	Macro F1
Adam	0,7515	0,792	0,809	0,800
SGD	0,6155	0,616	1,000	0,762

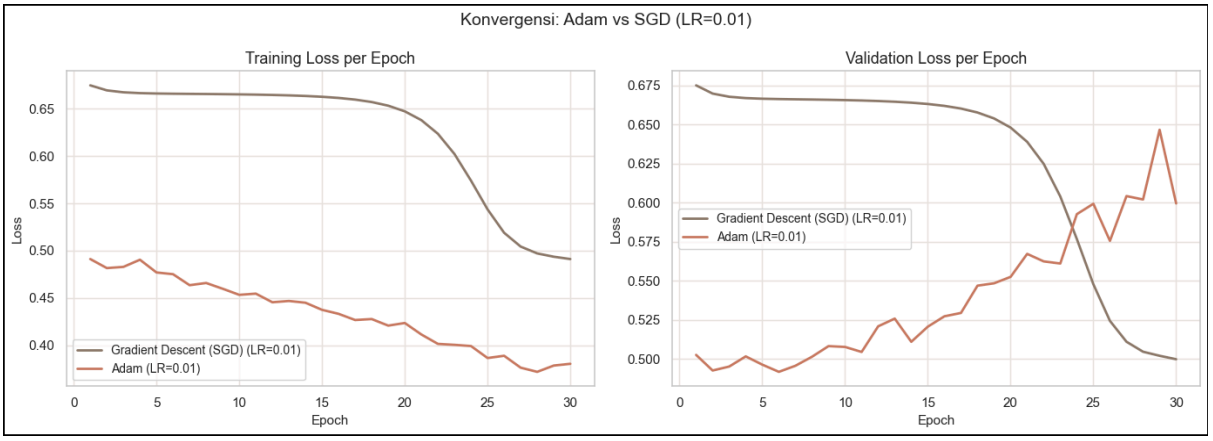


Gambar 2.18. Grafik *training loss* dan *validation loss* untuk setiap konfigurasi *optimizer* pada LR = 0.001

Berdasarkan kurva loss pada Gambar 2.18., Adam dengan LR = 0,001 berhasil konvergen dengan baik, menunjukkan penurunan *loss* yang stabil dari awal hingga akhir pelatihan. SGD dengan LR yang sama menghasilkan kurva yang sepenuhnya datar, mengonfirmasi bahwa pembaruan bobot berbasis SGD terlalu kecil untuk menghasilkan perubahan yang bermakna dalam 30 *epoch*. Hal ini menunjukkan keunggulan mekanisme adaptif Adam dalam kondisi *learning rate* kecil, di mana efektif *learning rate* per parameter dapat jauh lebih besar dari nilai global α yang ditetapkan.

Hasil pada LR = 0,01

Optimizer	Accuracy	Precision	Recall	Macro F1
Adam	0,7235	0,758	0,810	0,783
SGD	0,7510	0,785	0,820	0,802

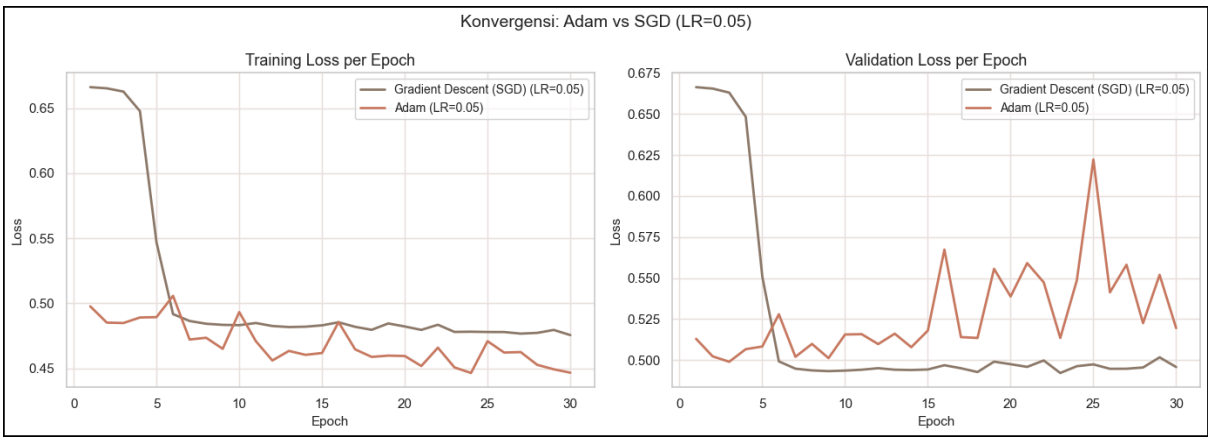


Gambar 2.19. Grafik *training loss* dan *validation loss* untuk setiap konfigurasi *optimizer* pada LR = 0.01

Berdasarkan kurva *loss* pada Gambar 2.19., SGD dengan LR = 0,01 menunjukkan konvergensi yang stabil dan menghasilkan *val loss* akhir yang lebih rendah dibanding Adam. Kurva Adam pada LR = 0,01 menunjukkan penurunan yang lebih cepat di awal, namun kemudian stagnan pada nilai *loss* yang lebih tinggi dibanding SGD. Hal ini mengindikasikan bahwa pada *learning rate* yang sudah optimal untuk SGD, mekanisme adaptif Adam justru menyebabkan efektif *learning rate* per parameter menjadi terlalu besar sehingga model melewati titik minimum.

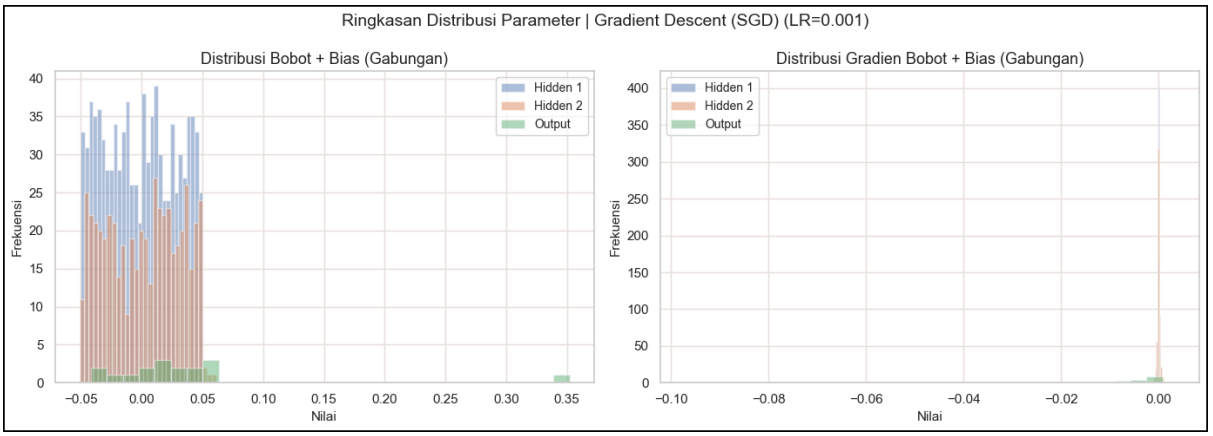
Hasil pada LR = 0,05

Optimizer	Accuracy	Precision	Recall	Macro F1
Adam	0,7425	0,754	0,864	0,805
SGD	0,7495	0,787	0,813	0,800

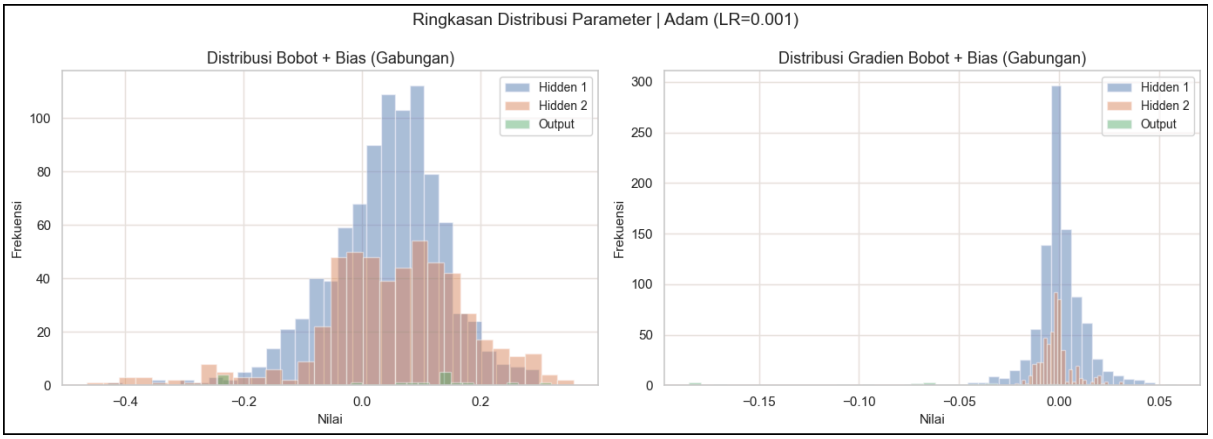


Gambar 2.20. Grafik *training loss* dan *validation loss* untuk setiap konfigurasi *optimizer* pada LR = 0.05

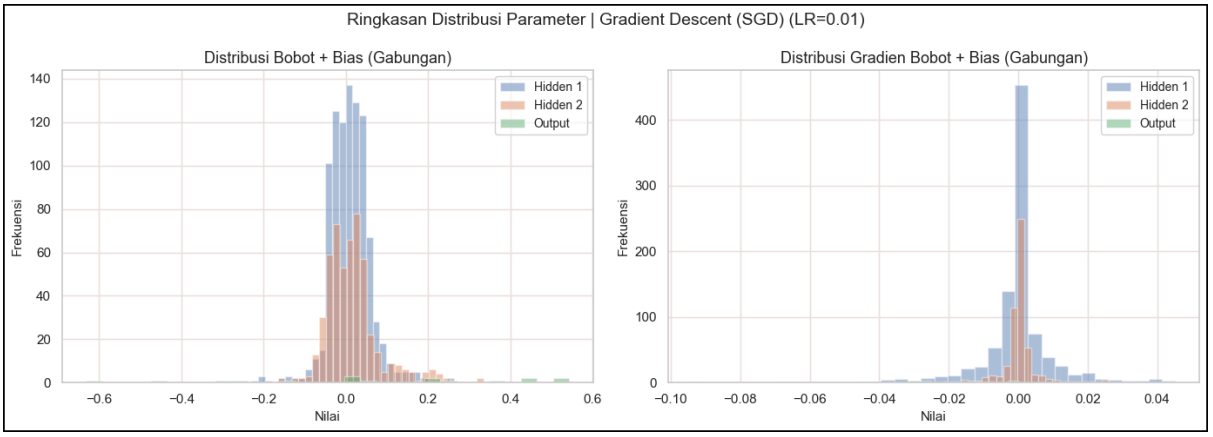
Berdasarkan kurva loss pada Gambar 2.20., kedua optimizer menunjukkan penurunan loss yang cepat di epoch-epoch awal. SGD dengan LR = 0,05 menunjukkan kurva yang sedikit lebih tidak mulus (osilasi lebih tinggi) dibanding Adam, yang konsisten dengan sifat SGD yang tidak memiliki mekanisme peredam osilasi. Adam pada LR = 0,05 menghasilkan kurva yang lebih mulus namun dengan val loss akhir yang sedikit lebih tinggi, mencerminkan kemungkinan adanya *overshooting* pada beberapa parameter akibat efektif learning rate yang terlalu besar.



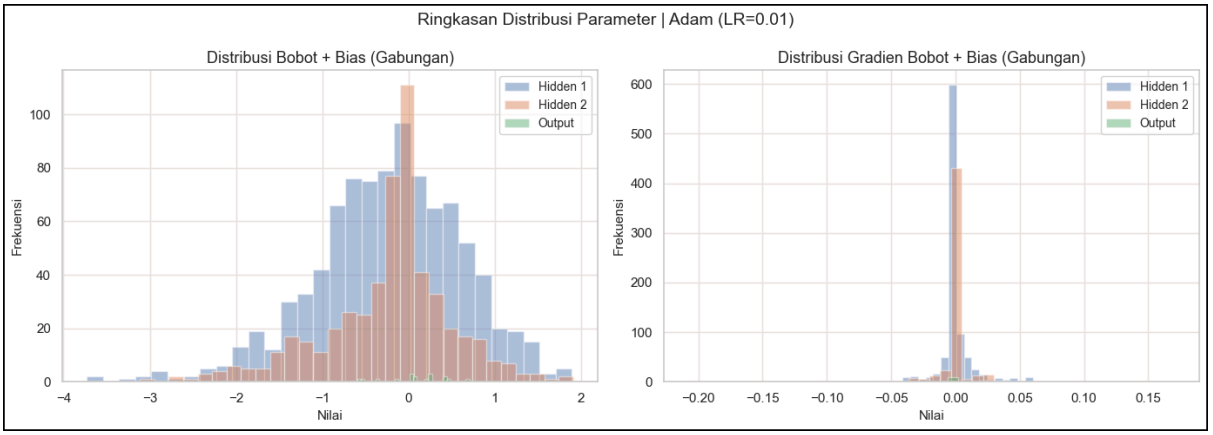
Gambar 2.21. Distribusi parameter optimizer SGD dengan LR = 0.001



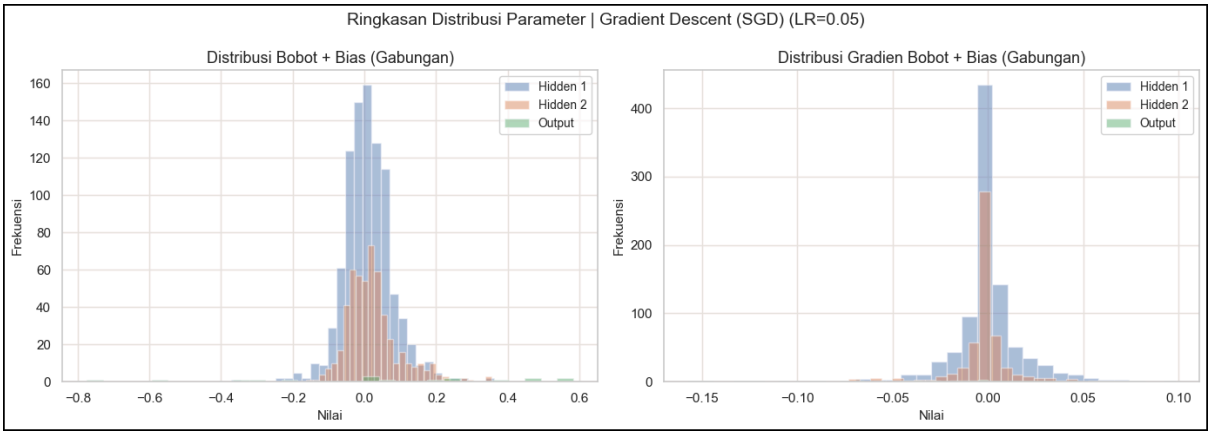
Gambar 2.22. Distribusi parameter optimizer Adam dengan LR = 0.001



Gambar 2.23. Distribusi parameter optimizer SGD dengan LR = 0.01



Gambar 2.24. Distribusi parameter optimizer Adam dengan LR = 0.01



Gambar 2.25. Distribusi parameter optimizer Adam dengan LR = 0.05

Berdasarkan distribusi bobot pada Gambar 2.21., SGD dengan LR = 0,001 menghasilkan distribusi yang hampir identik dengan inisialisasi awal (terpusat sempit di sekitar nol), mengonfirmasi minimnya pembaruan bobot. Adam dengan LR = 0,001 menghasilkan distribusi yang jauh lebih tersebar meski menggunakan *learning rate* global yang sama, yang secara visual menunjukkan efek adaptasi *learning rate* per parameter yang dimiliki Adam. Pada LR = 0,01 dan 0,05, distribusi bobot kedua *optimizer* terlihat lebih sebanding, namun Adam cenderung menghasilkan distribusi yang sedikit lebih simetris akibat normalisasi gradien yang dilakukannya.

Selanjutnya, SGD dengan LR = 0,001 menunjukkan distribusi gradien yang sangat terpusat di nol, mengonfirmasi bahwa meskipun gradien dihitung, dampak efektifnya terhadap pembaruan bobot tidak signifikan. Adam pada LR yang sama menunjukkan distribusi gradien yang jauh lebih menyebar, mencerminkan normalisasi gradien per parameter yang dilakukan Adam sehingga parameter dengan gradien kecil sekalipun tetap mendapat pembaruan yang bermakna. Pada LR = 0,05, SGD menunjukkan distribusi gradien dengan variansi lebih tinggi dibanding Adam, konsisten dengan perilaku osilasi yang terlihat pada kurva *loss*-nya.

Hasil analisis:

- Adam menunjukkan keunggulan yang jelas pada LR = 0,001 (Macro F1 = 0,800) dibanding SGD yang gagal total (Macro F1 = 0,762). Mekanisme adaptif Adam memungkinkan pembaruan bobot yang efektif meskipun learning rate global ditetapkan sangat kecil, terkonfirmasi dari distribusi bobot Adam yang jauh lebih tersebar dibanding SGD pada LR yang sama.
- SGD justru lebih unggul pada LR = 0,01 (Macro F1 = 0,802 vs 0,783). Pada *learning rate* yang sudah optimal untuk SGD, efektif *learning rate* per parameter Adam menjadi terlalu besar sehingga model melewati titik minimum (*overshooting*), tercermin dari val loss akhir Adam yang lebih tinggi pada kondisi ini.
- Adam menunjukkan **robustitas** yang lebih baik terhadap variasi *learning rate*. Adam berfungsi baik pada LR = 0,001 dan LR = 0,05 (Macro F1 0,800 dan 0,805), sementara SGD hanya mencapai performa optimal pada LR = 0,01. Hal ini menjadikan Adam sebagai pilihan yang lebih praktis apabila tuning *learning rate* tidak dilakukan secara ekstensif.
- Terdapat pertukaran (*trade-off*) yang konsisten antara kedua optimizer dimana SGD lebih unggul dalam Precision pada LR optimalnya, sedangkan Adam cenderung menghasilkan Recall yang lebih tinggi. Hal ini tercermin pada distribusi gradien Adam yang lebih merata di seluruh parameter.

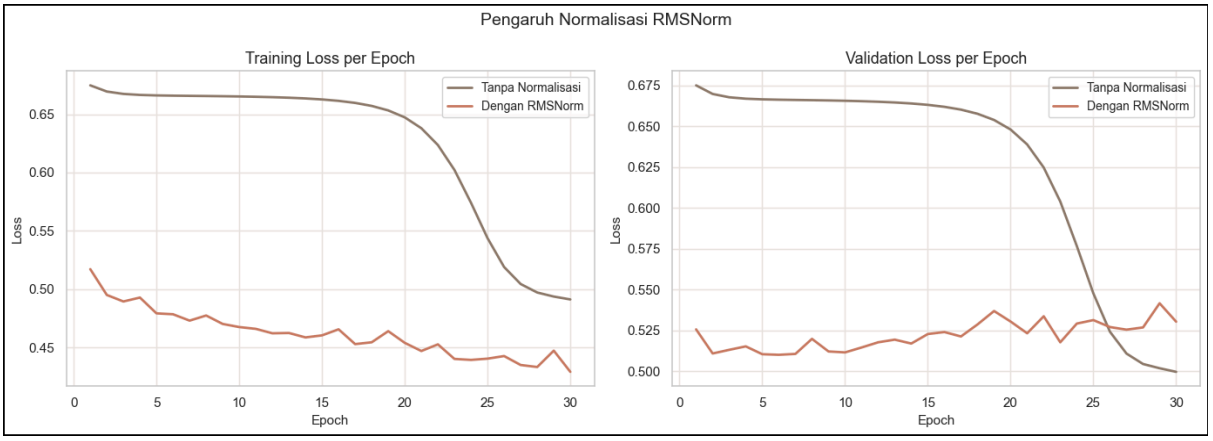
2.2.6. Pengaruh Normalisasi RMSNorm

Eksperimen membandingkan performa model dengan dan tanpa penerapan RMSNorm pada setiap hidden layer. RMSNorm bekerja dengan men-scale aktivasi setiap layer menggunakan nilai Root Mean Square-nya, sebagaimana dirumuskan berikut.

$$RMSNorm(x) = \frac{x}{RMS(x)} \cdot \gamma, \quad RMS(x) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}$$

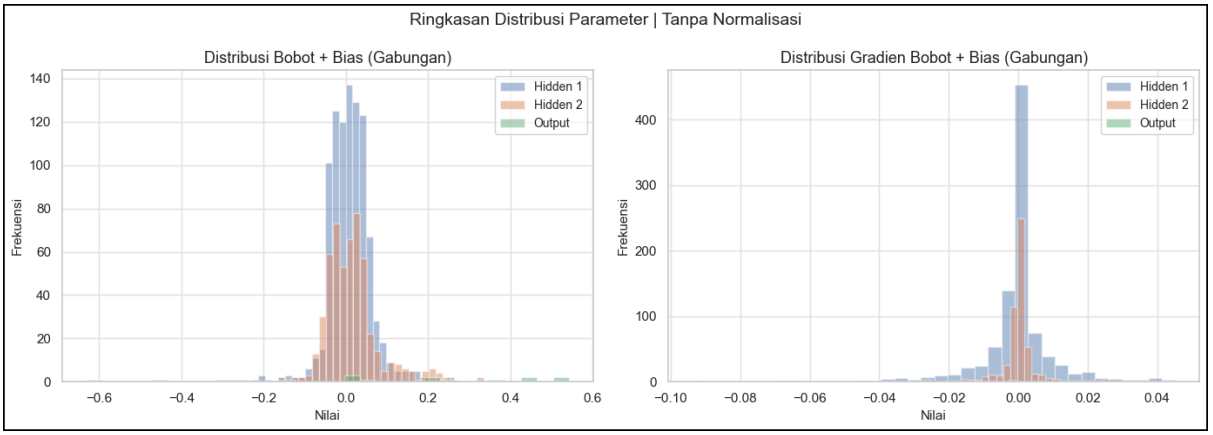
Di sini, γ merupakan parameter learnable scale yang perlu dioptimasi bersama bobot jaringan. Arsitektur yang digunakan adalah [32, 16] dengan aktivasi ReLU, learning rate = 0,01, batch size = 32, 30 epoch.

Konfigurasi	Accuracy	Precision	Recall	Macro F1
Tanpa Normalisasi	0,7510	0,785	0,820	0,802
RMSNorm	0,7305	0,778	0,787	0,782

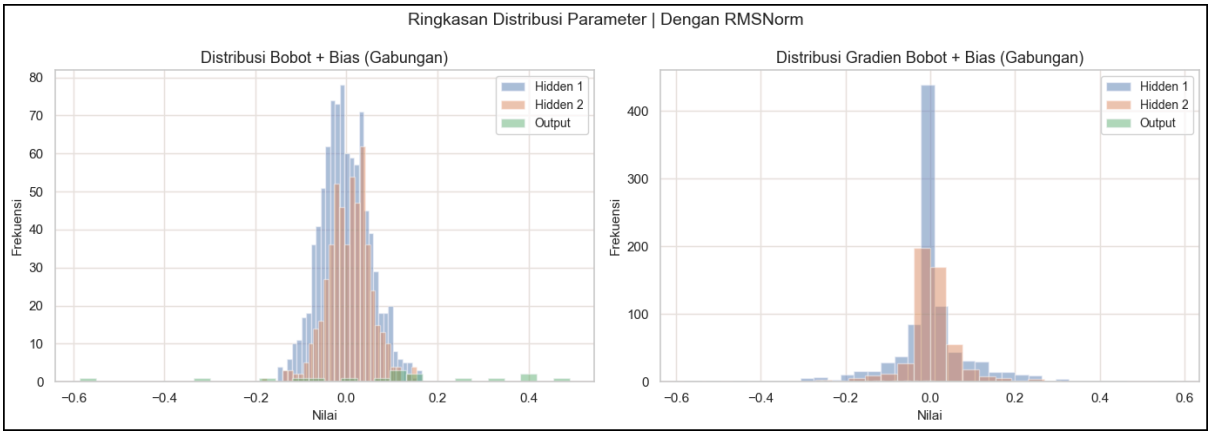


Gambar 2.26. Grafik *training loss* dan *validation loss* perbandingan RMSNorm

Berdasarkan kurva loss pada Gambar 2.26., konfigurasi tanpa normalisasi menunjukkan penurunan loss yang lebih tajam dan konvergensi yang lebih cepat dibandingkan konfigurasi dengan RMSNorm. Konfigurasi dengan RMSNorm menunjukkan penurunan loss yang lebih lambat dan cenderung stagnan di nilai yang lebih tinggi menjelang akhir pelatihan. Hal ini mengindikasikan bahwa proses normalisasi pada setiap layer menambah kompleksitas optimasi yang tidak dapat dikompensasi secara efektif dalam 30 epoch dengan arsitektur yang kecil.



Gambar 2.27. Distribusi parameter tanpa normalisasi



Gambar 2.28. Distribusi parameter dengan RMSNorm

Berdasarkan distribusi bobot pada Gambar 2.27. dan Gambar 2.28., konfigurasi tanpa normalisasi menghasilkan distribusi bobot yang lebih tersebar dan moderat. Konfigurasi dengan RMSNorm menunjukkan distribusi bobot yang lebih terpusat, yang mengindikasikan bahwa proses normalisasi membatasi rentang pembaruan bobot secara efektif. Hal ini terjadi karena RMSNorm secara aktif menstabilkan skala aktivasi antar layer, sehingga gradien yang merambat mundur memiliki magnitudo yang lebih seragam dan pembaruan bobot menjadi lebih konservatif.

Selanjutnya, konfigurasi dengan RMSNorm menunjukkan distribusi gradien yang lebih sempit dan terpusat dibandingkan konfigurasi tanpa normalisasi. Penstabilan skala aktivasi oleh RMSNorm menyebabkan gradien yang dihitung pada setiap layer menjadi lebih seragam, yang pada jaringan kecil justru mengurangi kemampuan model untuk melakukan pembaruan bobot yang diferensial antar layer. Pada jaringan yang lebih dalam, karakteristik ini justru bermanfaat karena mencegah exploding maupun vanishing gradient, namun pada jaringan dangkal seperti [32, 16], efeknya lebih bersifat membatasi.

Hasil analisis:

- Penerapan RMSNorm menghasilkan penurunan performa yang konsisten pada semua metrik. Macro F1 turun dari 0,802 menjadi 0,782, Recall turun dari 0,820 menjadi 0,787, dan kurva *loss* menunjukkan konvergensi yang lebih lambat dibanding konfigurasi tanpa normalisasi.
- Penurunan performa ini bukan disebabkan oleh kesalahan implementasi, melainkan oleh ketidaksesuaian konteks penggunaan. RMSNorm dirancang untuk memberikan

manfaat signifikan pada jaringan yang sangat dalam, di mana ketidakstabilan skala aktivasi antar layer menjadi masalah nyata. Pada jaringan dangkal [32, 16] dengan dataset berukuran sedang, masalah tersebut tidak terjadi secara berarti sehingga *overhead* dari parameter γ dan proses normalisasi tidak terkompensasi.

- Distribusi gradien yang lebih sempit pada konfigurasi RMSNorm mengonfirmasi bahwa normalisasi membatasi variasi pembaruan bobot antar layer. Pada jaringan kecil, pembatasan ini justru mengurangi ekspresivitas model dalam 30 epoch yang tersedia.
- Penerapan teknik normalisasi seperti RMSNorm sebaiknya dipertimbangkan secara kontekstual, terutama memperhatikan kedalaman jaringan, ukuran dataset, dan jumlah *epoch* pelatihan yang tersedia sebelum diterapkan.

2.2.7. Perbandingan dengan Library Scikit-learn

Perbandingan dilakukan antara implementasi FFNN from *scratch* dengan MLPClassifier dari sklearn menggunakan hyperparameter yang sama sedekat mungkin. Arsitektur: [32, 16], aktivasi ReLU, learning rate = 0,01, batch size = 32, 30 epoch.

Metrik	**FFNN from *scratch***	**Sklearn MLPClassifier**
Accuracy	0,7510	0,7565
Precision	0,785	0,784
Recall	0,820	0,833
Macro F1-Score	0,802	0,808

Hasil analisis:

- Implementasi FFNN from *scratch* menghasilkan performa yang sangat kompetitif, dengan selisih Macro F1 hanya 0,006 atau kurang dari 1% dibanding sklearn MLPClassifier.
- Precision implementasi FFNN from *scratch* (0,785) justru sedikit lebih tinggi dibanding sklearn (0,784), namun Recall-nya lebih rendah (0,820 vs 0,833). Hal ini mengindikasikan perbedaan dalam cara kedua model menyeimbangkan trade-off antara *Precision* dan *Recall*, yang dipengaruhi oleh perbedaan *optimizer* dan strategi inisialisasi bobot yang digunakan secara internal.
- Gap performa yang ada disebabkan oleh perbedaan konfigurasi internal sklearn. Sklearn MLPClassifier menggunakan optimizer Adam secara default yang mengadaptasi learning rate secara individual untuk setiap parameter, serta inisialisasi Xavier/Glorot yang menjaga skala gradien antar layer lebih baik dibandingkan Uniform[−0,1; 0,1]. Hal ini terkonfirmasi oleh hasil eksperimen optimizer pada Subbab 2.2.5, di mana Adam dengan LR = 0,001 berhasil belajar sementara SGD dengan LR yang sama gagal total.
- Implementasi FFNN *from scratch* terbukti mampu mereplikasi mekanisme inti jaringan saraf tiruan dengan tingkat akurasi yang mendekati implementasi produksi (*library* seperti *sklearn*). Gap yang tersisa dapat diperkecil dengan mengimplementasikan Adam optimizer dan inisialisasi Xavier/He.

Bagian 3. Kesimpulan dan Saran

3.1. Kesimpulan

Berdasarkan hasil implementasi dan eksperimen yang dilakukan, dapat disimpulkan beberapa hal sebagai berikut:

- Implementasi FFNN from *scratch* berhasil dilakukan menggunakan NumPy dengan komponen aktivasi, loss function, inisialisasi bobot, dan regularisasi yang modular.
- Performa model FFNN sangat bergantung pada pemilihan hyperparameter yang tepat, meliputi depth, width, fungsi aktivasi, learning rate, inisialisasi bobot, dan regularisasi.
- Penambahan width dari [16, 8] ke [32, 16] memberikan peningkatan performa yang signifikan (Macro F1 dari 0,762 ke 0,802), namun penambahan lebih lanjut ke [64, 32] tidak memberikan manfaat berarti, mengindikasikan bahwa kapasitas [32, 16] sudah memadai untuk merepresentasikan kompleksitas dataset ini. Penambahan depth dari 2 ke 3 hidden layer justru berdampak negatif akibat kombinasi masalah Dying ReLU dan vanishing gradient, terutama bila tidak disertai dengan strategi inisialisasi dan normalisasi yang tepat.
- Swish menghasilkan Macro F1 tertinggi (0,815) dengan Recall yang sangat tinggi (0,955) berkat sifatnya yang *smooth* dan *non-monotonic*. ReLU, Leaky ReLU, Linear, dan Tanh menghasilkan performa yang setara di kisaran Macro F1 0,800–0,803, sementara Sigmoid pada hidden layer menyebabkan kegagalan belajar total akibat vanishing gradient. Pemilihan fungsi aktivasi sebaiknya disesuaikan dengan prioritas metrik yang dibutuhkan aplikasi.
- Learning rate α = 0,01 memberikan keseimbangan terbaik antara kecepatan konvergensi dan stabilitas pembaruan bobot (Macro F1 = 0,802). Learning rate terlalu kecil (α = 0,001) menyebabkan model tidak mampu bergerak signifikan dari titik inisialisasi dalam 30 epoch, sedangkan learning rate terlalu besar (α = 0,05) berpotensi menyebabkan overshooting di sekitar nilai minimum sehingga model tidak mencapai titik optimal yang sama.
- L2 dengan λ = 0,001 menghasilkan Macro F1 terbaik (0,803) melalui efek weight decay yang sedikit meningkatkan generalisasi tanpa mengorbankan kapasitas model. L1 dengan nilai λ yang sama menyebabkan kegagalan belajar total akibat weight collapse yang dipicu oleh interaksi negatif antara penalti L1, fungsi aktivasi ReLU, dan inisialisasi Uniform, sehingga penggunaannya memerlukan nilai λ yang jauh lebih kecil atau teknik mitigasi tambahan.
- Adam menunjukkan keunggulan dalam hal robustitas terhadap variasi learning rate, khususnya pada learning rate kecil (Macro F1 = 0,800 pada α = 0,001) di mana SGD gagal total. Namun, SGD lebih unggul pada learning rate yang sudah dikalibrasi dengan baik (Macro F1 = 0,802 pada α = 0,01). Adam menjadi pilihan yang lebih praktis apabila tuning learning rate tidak dilakukan secara ekstensif.
- Penerapan RMSNorm pada jaringan dangkal [32, 16] menghasilkan penurunan performa dibanding konfigurasi tanpa normalisasi (Macro F1 turun dari 0,802 ke 0,782). Hal ini disebabkan oleh ketidaksesuaian konteks penggunaan, di mana overhead parameter γ dan proses normalisasi tidak terkompensasi pada jaringan kecil yang tidak mengalami masalah ketidakstabilan skala aktivasi secara berarti.
- Implementasi FFNN from *scratch* terbukti mampu mereplikasi mekanisme inti jaringan saraf tiruan secara kompetitif, dengan selisih Macro F1 hanya 0,006 atau

kurang dari 1% dibanding sklearn MLPClassifier (0,802 vs 0,808). Gap yang tersisa disebabkan oleh perbedaan konfigurasi internal *sklearn* yang menggunakan Adam optimizer secara default serta inisialisasi Xavier/Glorot, dan dapat diperkecil dengan mengadopsi konfigurasi serupa pada implementasi *from scratch*.

3.2. Saran

- Implementasi FFNN dapat ditingkatkan dengan menambahkan optimasi seperti adaptive learning rate (Adam, RMSProp) untuk mempercepat konvergensi.
- Penambahan *batch normalization* dapat membantu stabilisasi pelatihan terutama untuk jaringan yang lebih dalam.
- Early stopping dapat diimplementasikan untuk mencegah *overfitting* secara otomatis.
- Eksperimen dengan variasi hyperparameter yang lebih luas dapat dilakukan untuk menemukan konfigurasi optimal pada dataset ini.

Pembagian Tugas

NIM	Tugas
18223120	• Mengimplementasikan Leaky ReLU, Swish • Mengimplementasikan Xavier initialization dan He initialization. • Mengimplementasikan metode normalisasi RMSNorm • Mengimplementasikan optimizer Adam • Melakukan Analisis pada Notebook • Menulis Laporan
18223123	• Mengimplementasikan struktur utama model FFNN (forward propagation, backward propagation, dan alur training mini-batch) • Melakukan Analisis pada Notebook • Menulis Laporan
18223129	• Mengimplementasikan struktur utama model FFNN (arsitektur layer, forward propagation, backward propagation, dan alur training mini-batch) • Mengimplementasikan komponen inti sesuai spesifikasi (fungsi aktivasi dan loss, inisialisasi bobot, regularisasi, visualisasi distribusi bobot/gradien, serta save/load model) • Mengimplementasikan struktur utama notebook pengujian dan pipeline eksperimen • Menulis laporan • Melakukan analisis pada notebook

Referensi

- 3Blue1Brown. (2025, 18 Februari). Neural networks [Playlist]. YouTube. https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- Scikit-learn documentation: MLPClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
- NumPy documentation. https://numpy.org/doc/2.2/
- Global Student Placement & Salary Dataset.

Lampiran Form Penggunaan AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Leonard Arif Sutiono 18223120
Harfhan Ikhtiar Ahmad R 18223123
Izhar Alif Akbar 18223129

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi

Kode Mata Kuliah : IF3270

Judul Tugas/Proposal : IF3270 Pembelajaran Mesin Laporan Tugas Besar 1 - Feedforward Neural Network

Menyatakan bahwa saya/kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Digunakan untuk pembuatan *helper function* pada notebook dengan detail: ● Pembuatan kustom visualisasi grafik agar lebih mudah dipahami. ● Pembuatan template output untuk metrik evaluasi dan analisis.
Bantuan Diskusi dalam Penyusunan Konten:	Ya	Digunakan untuk membantu *brainstroming*

menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.		terkait struktur repositori.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:	-	-

Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 18 Maret 2026

Harfhan Ikhtiar A. R.	Izhar Alif Akbar	Leonard Arif Sutiono
18223129	18223129	18223120